

Département d'informatique 420-126-LI - Introduction à la programmation	Note de cours Semaine 2
Écrites par Robert Aubé	Modifiées par Julien Brunet

Table des matières

Types primitifs ou élémentaires de variables	2
Les opérateurs.....	3
Opérateurs arithmétiques	3
Opérateur modulo %	3
Opérateurs d'assignation.....	4
Opérateurs d'incrémentement (++) et de décrémentation (--).....	5
Opérateurs relationnels.....	6
Priorité des opérateurs	6
Lire une donnée au clavier fournie par l'utilisateur (Scanner).....	7
Lire une donnée au clavier pour une valeur (variable) de type String	7
Lire une donnée au clavier pour une valeur (variable) de type Entier (int)	8
Introduction à la portée des variables	9
Constantes	10

Types primitifs ou élémentaires de variables

Les types primitifs de variables sont utilisés pour placer en mémoire les données de base les plus courantes. Ces types spécifient quelle donnée pourra être conservée dans l'espace réservé. Chacun de ses types est caractérisé par un nombre d'octets prédéfinis. Les types élémentaires commencent tous par une minuscule.

Voici un tableau répertoriant les huit types primitifs du langage Java :

Type de donnée	Type primitif	Description
Entiers	byte	1 octet -128 à +127
	short	2 octets -32768 à +32767
	int	4 octets -2147483648 à +2147483647
	long	8 octets -9223372036854775808 à 9223372036854775807
Flottants ou décimaux	float	4 octets -1.4×10^{-45} à $+3.4 \times 10^{38}$ ou 2^{-1074} à 2^{1024} Par défaut, les variables sont créées en double . Pour forcer float , il faut faire suivre la valeur par la lettre f ou F . Exemple <code>float x;</code> <code>x = 12.5;</code> //erreur ici 12.5 est de type double <code>x = 12.5f;</code> //ok
	double	8 octets 4.9×10^{-324} à $+1.7 \times 10^{308}$ ou 2^{-1074} à 2^{1024}
Caractère	char	2 octets 'c' (unicode) Exemple <code>char c = 'e';</code>
Booléen	boolean	1 octet true et false Exemple <code>boolean estValide = true;</code>

Il y a aussi le type « **String** » qui n'est pas un type primitif ou élémentaire, mais plutôt une **classe** qui contient des « **char** ». Comme toute classe, le type « **String** » s'écrit avec une lettre majuscule contrairement aux types de base. Les classes et les « **String** » seront vus plus tard dans la session...

Les opérateurs

- Les opérateurs sont des symboles qui effectuent une action spécifique sur une ou deux opérandes et qui retournent un résultat. On dira que le résultat « **est la valeur de l'expression** ».

Exemple 1

Dans :

1 + 6

«1», «6» sont des **opérandes**

«+ » est l'opérateur

7 « est la valeur de cette expression ».

Exemple 2

Dans :

++a

«a» est l'**opérande**

«++» est l'opérateur

« La valeur de cette expression » est (« la valeur de a » + 1).

- En Java, il existe plusieurs types d'opérateurs, pour le moment, nous allons nous intéresser aux catégories suivantes :
 - arithmétiques
 - d'assignation
 - d'incrémentatation et de décrémentatation
 - relationnels

Opérateurs arithmétiques

opérateur	signification	expression	valeur
+	addition	2 + 2	4
-	soustraction	3 - 2	1
*	multiplication	3 * 2	6
/	division	4 / 2	2
%	Modulo (reste, en unité, de la division)	12 % 5	2

Opérateur modulo %

Exemples

expression	valeur
6 % 5	1
6 % 0	Erreur division par 0
6 % 4	2
1 % 2	1
2 % 3	2
125 % 128	125
5.5 % 2	1.5

Opérateurs d'assignation

- Ces opérateurs permettent d'affecter une valeur à une variable.
- Nous avons vu l'affectation simple avec le symbole « `=` », il en existe d'autres qui permettent de simplifier des opérations arithmétiques et d'affectation.

Opérateurs d'assignation		Exemple	Valeur de a
=	Affectation usuelle	<code>a = 2 + 2 ;</code>	4
+=	Additionne deux valeurs et stocke le résultat dans la variable (à gauche)	<code>a = 4 ;</code> <code>a += 3 ;</code> identique à <code>a = a + 3 ;</code>	7
-=	Soustrait deux valeurs et stocke le résultat dans la variable	<code>a = 4 ;</code> <code>a -= 3 ;</code> identique à <code>a = a - 3 ;</code>	1
*=	Multiplie deux valeurs et stocke le résultat dans la variable	<code>a = 3;</code> <code>a *= 2;</code> identique à <code>a = a * 2;</code>	6
/=	Divise deux valeurs et stocke le quotient dans la variable	<code>a = 4</code> <code>a /= 2;</code> identique à <code>a = a / 2;</code>	2
%=	Divise deux valeurs et stocke le reste entier de la division dans la variable	<code>a = 4</code> <code>a %= 3</code> identique à <code>a = a % 3;</code>	1

Opérateurs d'incrémentation (++) et de décrémentation (--)

- L'incrémentation permet l'ajout d'une unité à une variable. Par exemple si une variable qui vaut 4 aura la valeur 5 après l'avoir incrémenté de 1.
- À l'inverse, la décrémentation diminue une d'une unité une variable.
- En Java, nous disposons de deux opérateurs « ++ » et « -- » pour respectivement incrémenter ou décrémentation.
 - ++i ou i++ pour la « pré » ou « post » incrémentation
 - --i ou i-- pour la « pré » ou « post » décrémentation
- Employé seul, il n'y a pas de différence entre l'effet de la notation :
 - préfixe (++i, --i) et la notation postfixe (i++, i--).
- Incrémenter/décrémenter une variable dans une expression et en même temps affecter sa valeur à une autre variable. Dans ce cas, nous devons choisir entre la notation **préfixe** et **postfixe**:
 - x = ++i (préfixe) incrémente d'abord et passe la valeur incrémentée à x
 - x = --i (préfixe) décrémente d'abord et passe la valeur décrémentée à x
 - x = i++ (postfixe) passe d'abord la valeur de i à x et incrémente après
 - x = i-- (postfixe) passe d'abord la valeur de i à x et décrémente après
- Exemples dans une expression:

Opérateur	Signification	Exemple	Fonctionnement	Valeur de « a » après l'affectation	Valeur de « b » après l'affectation
++	incrément	a = 2; b = ++a;	Incrémente d'abord a et passe la valeur incrémentée à b	3	3
		a = 2; b = a++;	Passe d'abord la valeur de a à b et incrémente après	3	2
--	décrément	a = 2; b = --a;	Décrémente d'abord a et passe la valeur décrémentée à b	1	1
		a = 2; b = a--;	Passe d'abord la valeur de a à b et décrémente après	1	2
Expressions plus complètes					
++	incrément	a = 2; b = ++a + 2;	Incrémente d'abord a, additionne 2 et passe le résultat à b	3	5
		a = 2; b = a++ + 2;	Additionne 2 à a, passe le résultat à b et incrémente a après	3	4
--	décrément	a = 2; b = --a + 2;	Décrémente d'abord a, additionne 2 et passe le résultat à b	1	3
		a = 2; b = a-- + 2;	Additionne 2 à a, passe le résultat à b et décrémente a après	1	4

Opérateurs relationnels

- Java propose une série d'opérateurs d'égalité et de comparaison qui renvoient comme résultat un booléen vrai ou faux (true ou false).
- On les utilise notamment **pour prendre des décisions** dans un programme comme on verra à la leçon suivante.

opérateur	signification	exemple	Valeur de l'expression
<code>==</code>	égal	<code>4 == 4</code>	true
<code>!=</code>	différent	<code>4 != 4</code>	false
<code>></code>	plus grand	<code>4 > 4</code>	false
<code>>=</code>	plus grand ou égal	<code>4 >= 4</code>	true
<code><</code>	plus petit	<code>4 < 4</code>	false
<code><=</code>	plus petit ou égal	<code>4 <= 4</code>	true

Priorité des opérateurs

Dans tous les langages de programmation, il existe un ordre d'exécution des calculs spécifiques. Les calculs sont analysés et réalisés selon la **priorité des opérations** et ensuite ils sont évalués **de la gauche vers la droite**.

Ce tableau présente sous une forme simplifiée l'ordre des priorités des opérations les plus courantes.

Priorité	Opérateur
1	<code>() []</code>
2	<code>++ -- !</code> unaire (comme <code>++i</code> , <code>--i</code> et le non (not) <code>!a</code>)
2	<code>le -</code> unaire (changement de signe exemple : <code>-a</code>)
3	<code>*</code> <code>/</code> <code>%</code>
4	<code>+</code> <code>-</code>
5	<code><</code> <code>></code> <code><=</code> <code>>=</code>
6	<code>==</code> <code>!=</code>
7	<code>&&</code>
8	<code> </code>
9	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code>^=</code> <code> =</code> <code><<=</code> <code>>>=</code>
10	<code>,</code>

Exemple

Pour vous aider à comprendre, observons les opérations suivantes :

```
int a = 0, b = 0, c = 0, d = 0;
a = 5;
b = 10;
c = 2;

d = a + b * c;
// Le compilateur fera (b * c) + a = 25
d = c * b / 2;
// le compilateur fera c * b / 2 = 10
d = (a + b) * c;
// Le compilateur fera (a + b) * c = 30
d = c * (b / 2);
// Le compilateur fera (b / 2) * c = 10
```

Note importante et Bonne pratique : Pour la clarté et afin de lever les ambiguïtés, les parenthèses « `()` » demeurent le meilleur moyen de composer vos expressions mathématiques.

Lire une donnée au clavier fournie par l'utilisateur (**Scanner**)

- Pour lire une donnée fournie par l'utilisateur, le code devra obligatoirement utiliser une **variable** pour « stocker » la donnée pour être en mesure d'utiliser cette valeur par la suite.

Lire une donnée au clavier pour une valeur (variable) de type **String**

Exemple

```
//Documentation de la classe Scanner:
//https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Scanner.html

import java.util.Scanner; //Librairie nécessaire pour Lire des caractères

..../**
....* Lit au clavier un nom et un prénom et produit un affichage qui retourne
....* Les valeurs saisies au clavier
....*/
public void lireEtAfficherNomPrenom() {
    Scanner sc = null; //Variable pointeur (référence) pour l'utilitaire de lecture
    String nom = "";
    String prenom = "";

    sc = new Scanner(System.in); //Instanciation de l'utilitaire de lecture

    System.out.println("Entrez votre nom : ");

    //Appel d'une méthode de l'objet Scanner qui récupère une chaîne de caractères
    //saisies au clavier.
    nom = sc.nextLine(); //Lecture du nom, lire du texte

    System.out.println("Entrez votre prénom : ");
    prenom = sc.nextLine(); //Lecture du prénom, lire du texte

    System.out.println("Bonjour Monsieur ou Madame " + nom);
    System.out.print("Je sais que votre prénom est " + prenom);

}
```

À l'exécution de cette portion de code, voici ce qui sera affiché (**En considérant que Fortin et Denis ont été saisis au clavier**) :

```
Entrez votre nom :
Fortin
Entrez votre prénom :
Denis
Bonjour Monsieur ou Madame Fortin
Je sais que votre prénom est Denis
```

Lire une donnée au clavier pour une valeur (variable) de type Entier (**int**)

Exemple

```
/**
 * Saisit le nombre de billes que possèdent Denis et Louise
 * et affiche le total de leurs billes.
 */
public void lireEtAfficherDesNombresDeBilles() {
    Scanner sc = null; //Variable pointeur (référence) pour l'utilitaire de lecture
    int nbBillesDenis = 0;
    int nbBillesLouise = 0;
    int totalBilles = 0;

    sc = new Scanner(System.in); //Instanciation de l'utilitaire de lecture

    System.out.println("Entrez le nombre de billes de Denis : ");
    nbBillesDenis = sc.nextInt(); //Lire un entier

    System.out.println("Entrez le nombre de billes de Louise : ");
    nbBillesLouise = sc.nextInt(); //Lire un entier

    totalBilles = nbBillesDenis + nbBillesLouise;

    System.out.println("Denis et Louise ont " + totalBilles + " billes ensembles.");
}
```

À l'exécution de cette portion de code, voici ce qui sera affiché (*En considérant que 10 et 24 ont été saisis au clavier*) :

```
Entrez le nombre de billes de Denis :
10
Entrez le nombre de billes de Louise :
24
Denis et Louise ont 34 billes ensembles.
```

Introduction à la portée des variables

- Selon l'endroit où l'on déclare une variable, celle-ci pourra être accessible (visible) de partout dans le code ou bien uniquement dans une portion de celui-ci, on parle alors de portée d'une variable.
- En java, toute variable est déclarée à l'intérieur d'un groupe d'accolades « { } » (bloc de code). Sa portée se restreint donc à l'intérieur de ce bloc de code. Le groupe d'accolades peut être une classe, une méthode ou d'autres structures que nous verrons bientôt (comme **for** et **if** que vous voyez ci-dessous encore inconnus à ce stade)
- **Important** : Aussi, une variable est disponible pour utilisation uniquement après qu'elle a été déclarée.
- **Important** : Il est interdit d'avoir deux variables de même nom si elles ont une portée commune.

Exemple

```
10 > public class Exemple4_PorteeVariables {
11
12 >     public static void main(String[] args) {
13         //a n'est valide que dans le main
14         int a = 3;
15         //int a = 13; //pour le if plus bas
16         Exemple4_PorteeVariables objet = new Exemple4_PorteeVariables();
17         System.out.println("Voici a : " + a );
18
19         objet.methode1();
20         objet.methode2();
21
22         //b et c sont inconnus ici!
23
24         if (a > 3){
25             int d = 10;
26             a += d;
27             System.out.println("Voici d : " + d );
28             System.out.println("Voici a : " + a );
29
30             //b, c et e sont inconnus ici!
31         }else{
32             int e = 10;
33             a -= e;
34             System.out.println("Voici e : " + e );
35             System.out.println("Voici a : " + a );
36
37             //b, c et d sont inconnus ici!
38         }
39     }
40
41     public void methode2() { 1 usage
42         //b n'est valide que dans cette methode
43         int b = 5;
44         b += 8;
45         System.out.println("Voici b : " + b );
46         //a et c sont inconnus ici!
47     }
48
49     public void methode1() { 1 usage
50         //c n'est valide que dans cette methode
51         int c = 7;
52         System.out.println("Voici c : " + c );
53
54         // a et b sont inconnus ici!
55     }
56 }
```

Nom	Type	Variable	No de ligne de la portée
a	int	variable	14 à 39
b	int	variable	43 à 47
c	int	variable	51 à 55
d	int	variable	25 à 31
e	int	variable	32 à 38

Constantes

Une constante est une donnée dont la valeur est **inchangeable** lors de l'exécution d'un programme. Elle est initialisée lors de sa création et sa valeur ne changera jamais tout au long du programme. Une constante peut être définie pour la classe entière ou localement dans une méthode. Voici comment déclarer et **initialiser une constante locale**:

```
final double TAUX_TAXE = 0.075;
```

Par convention, et pour les distinguer des variables, les constantes sont écrites **entièrement en majuscules** et **chaque mot qui la compose est séparé par un souligné « _ »**.

Afin qu'elle soit accessible par toutes les méthodes, une constante est souvent définie au niveau de la classe (plus globale). Dans ce cas elle est en plus être déclarée « **public** » et « **static** ».

```
public static final double TAUX_TAXE = 0.075; //Au niveau de la classe
```

NOTE : On privilégiera cette deuxième manière de définir les constantes.